**PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL**
**FORMATO GUÍA DE APRENDIZAJE – ACTIVIDADES**
**ESTRUCTURAR LA PROPUESTA TÉCNICA**

# Actividad

## Construcción de una API REST con Node.js, Express y MySQL


**Presentado a:** Yoly Alexandra Guevara Muños
**Por Aprendiz:** Jenny Carolina Marrugo Ussa
**Ficha:** 3312932



Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

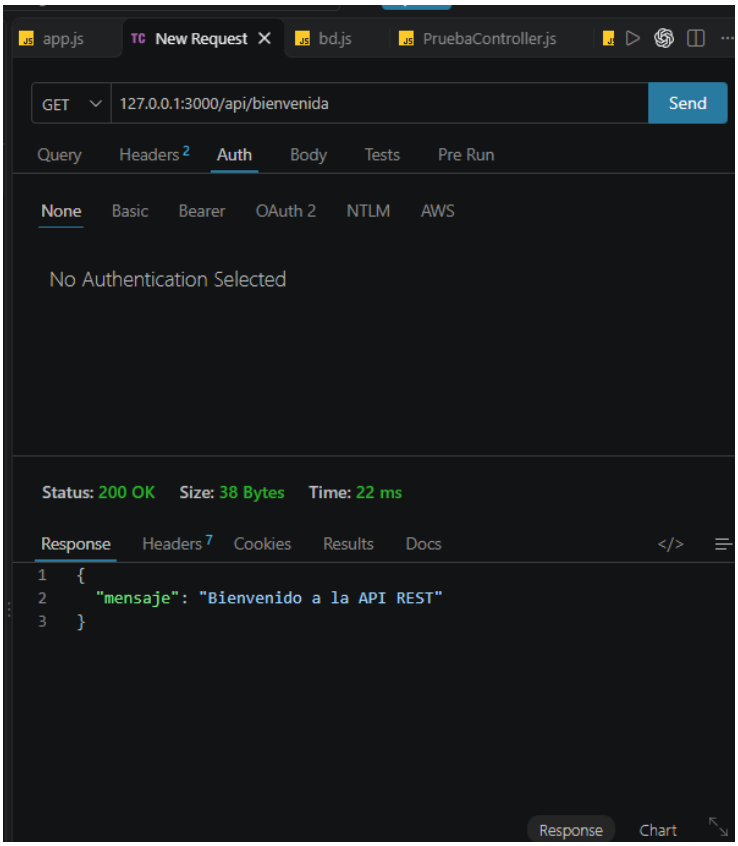
Popayán, día 08 de noviembre del año 2026

## Introducción

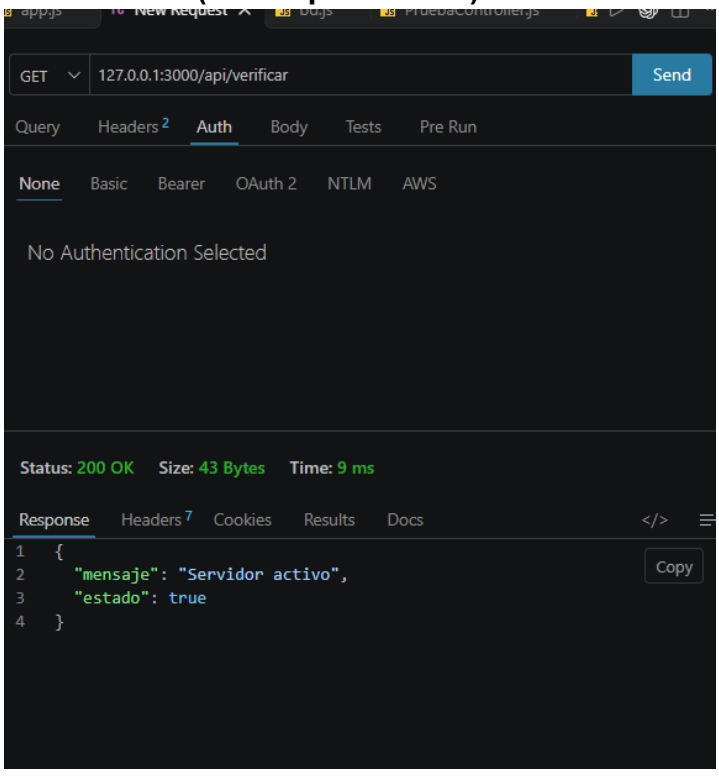
En este trabajo se presenta el desarrollo de una práctica enfocada en la creación de una API REST utilizando herramientas como Node.js, Express y una base de datos en MySQL. El objetivo principal de este ejercicio es aplicar de forma práctica los conceptos vistos en clase, organizando el código mediante clases, controladores y rutas separadas. Con esto se busca entender cómo funciona la comunicación entre el servidor y la base de datos al momento de registrar y consultar información, logrando así un funcionamiento correcto y comprobado a través de pruebas.

1. **Evidencias de Pruebas en Thunder Client (Pantallazos)**
   A. **Módulo de Pruebas (/api)**

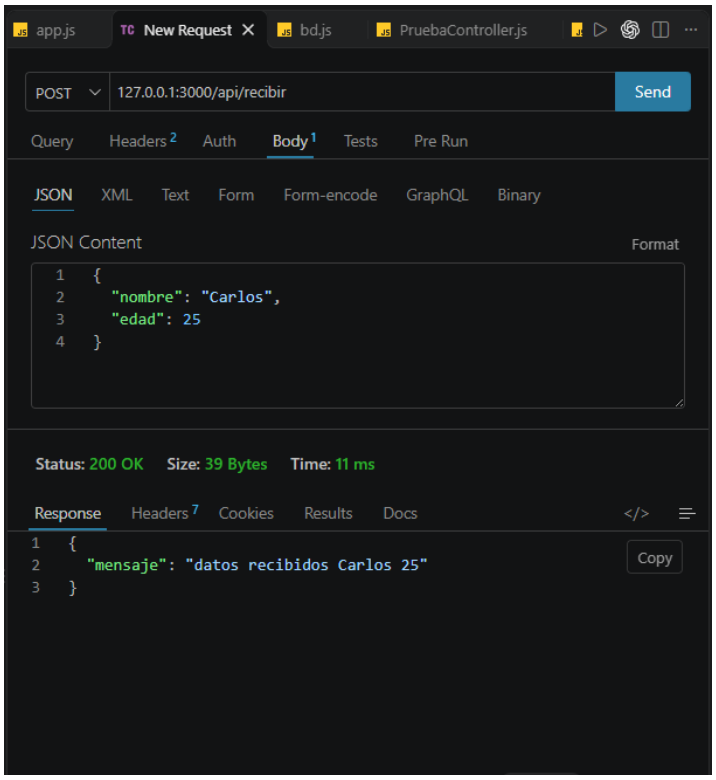
**Bienvenida (GET /api/bienvenida):**



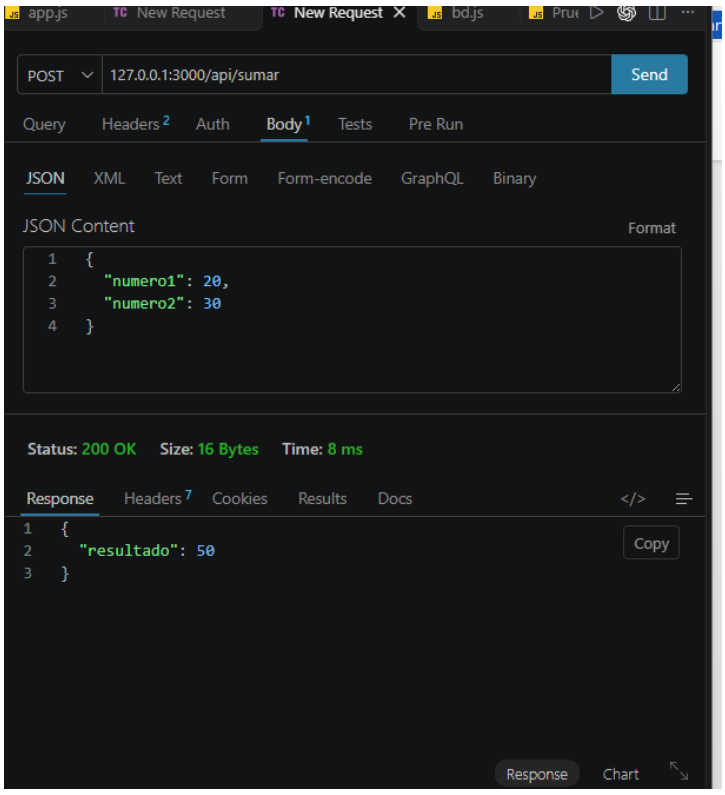

**Verificar Servidor (GET /api/verificar):**



## Recibir Datos (POST /api/recibir):

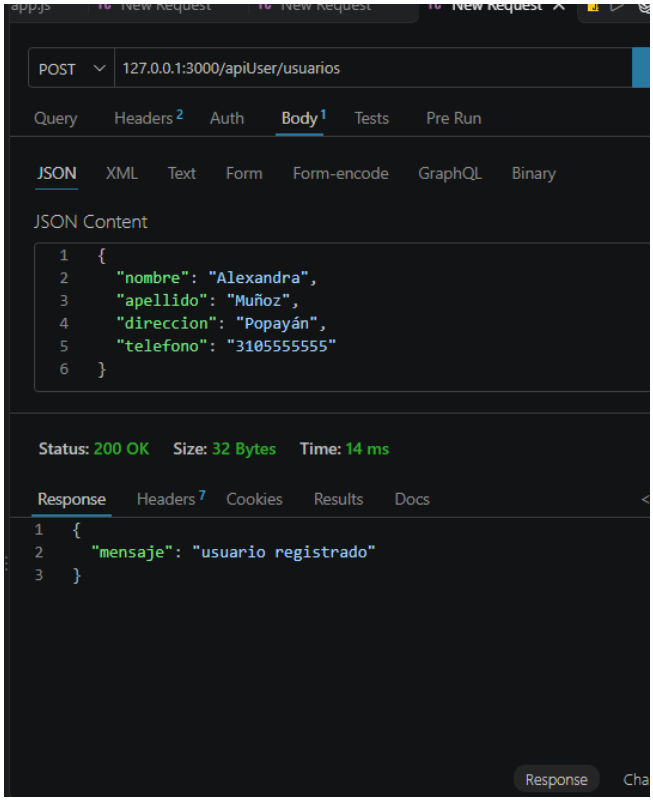



## Sumar Dos Números (POST /api/sumar):

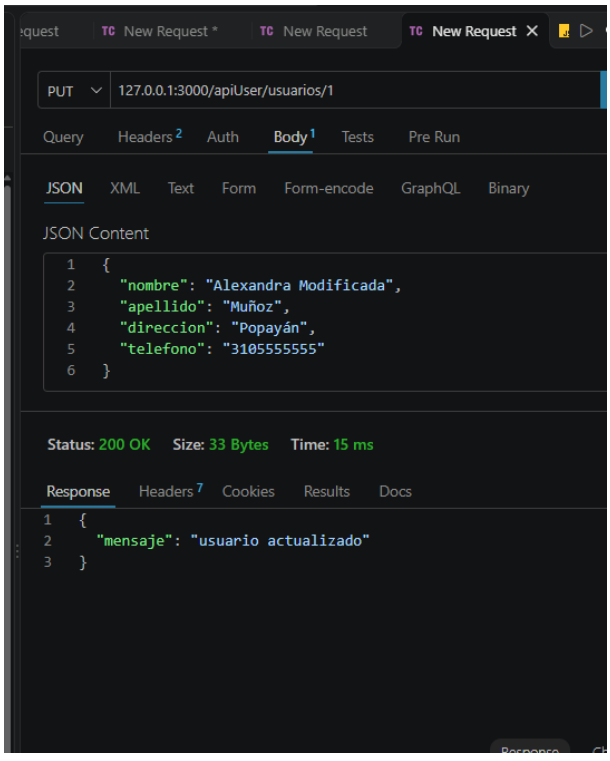


## B. Módulo de Usuarios (/apiUser)

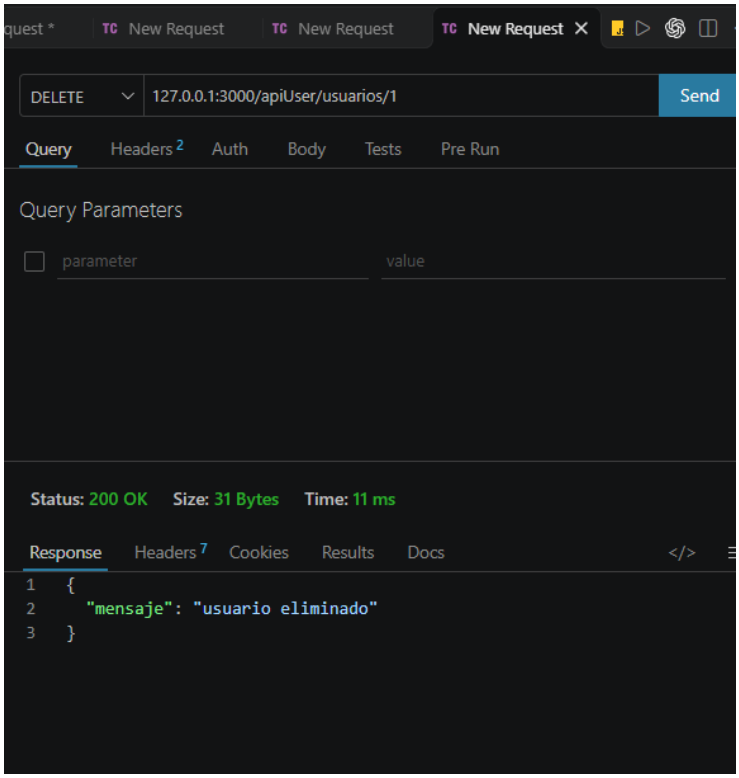
### Registrar Usuario (POST /apiUser/usuarios):




### Consultar Todos (GET /apiUser/usuarios):

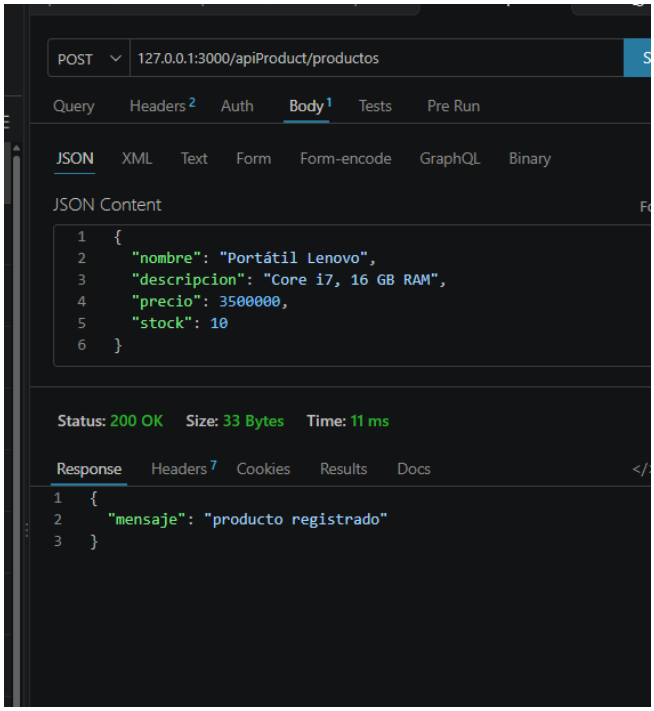


### Eliminar Usuario (DELETE /apiUser/usuarios/:id):

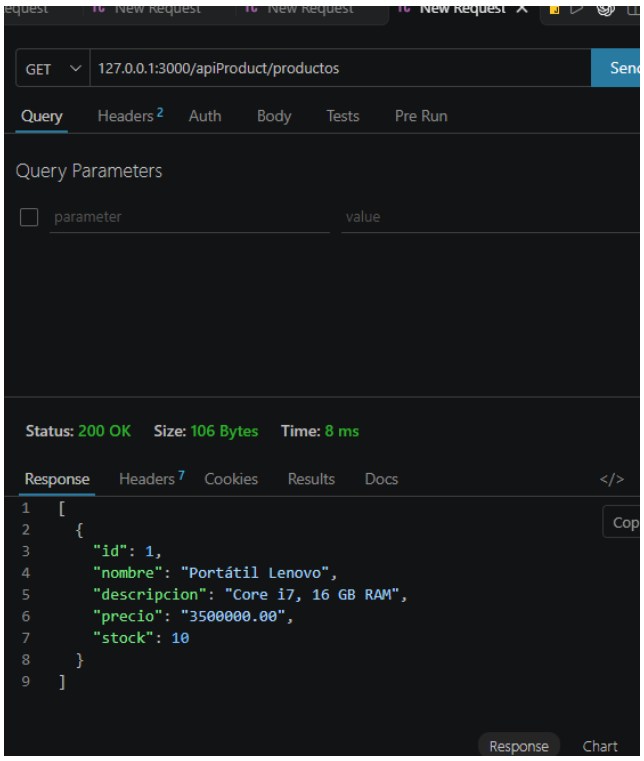



## C. Módulo de Productos (/apiProduct)

### Registrar Producto (POST /apiProduct/productos):

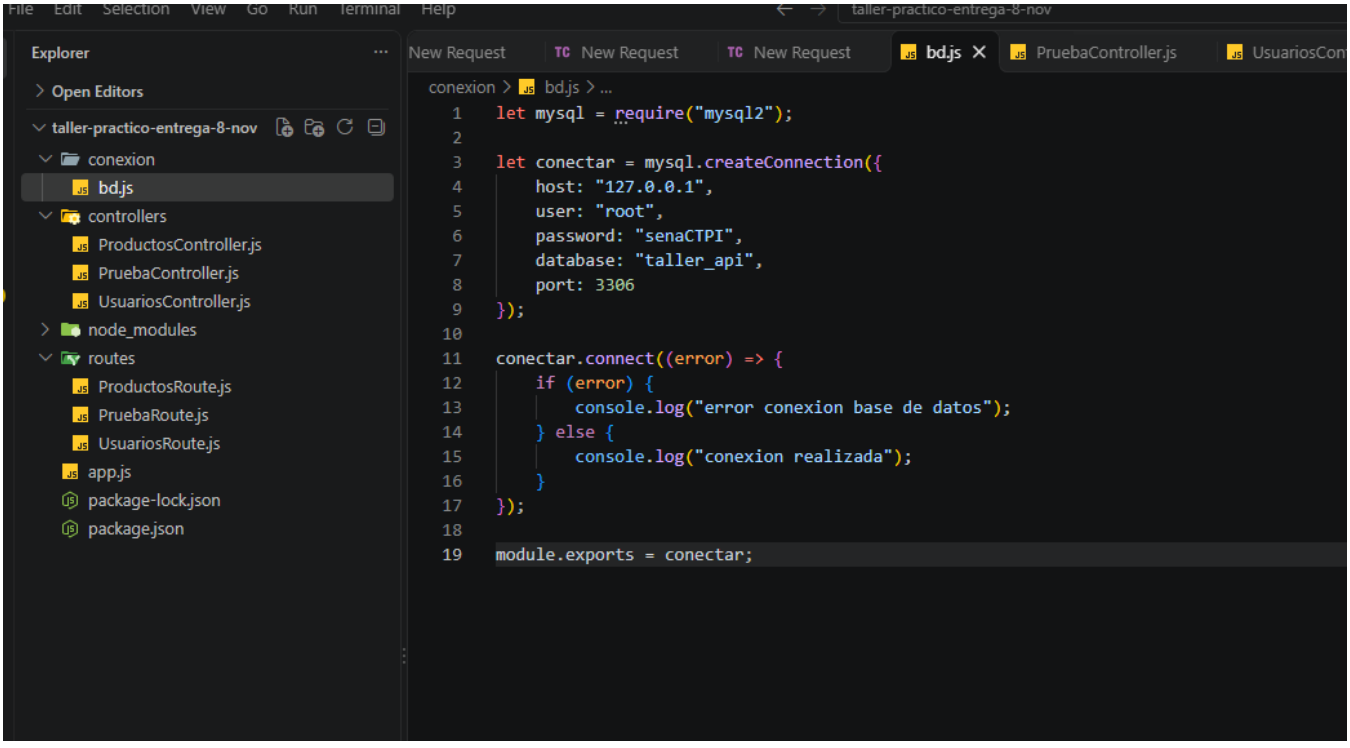


## Consultar Todos (GET /apiProduct/productos):



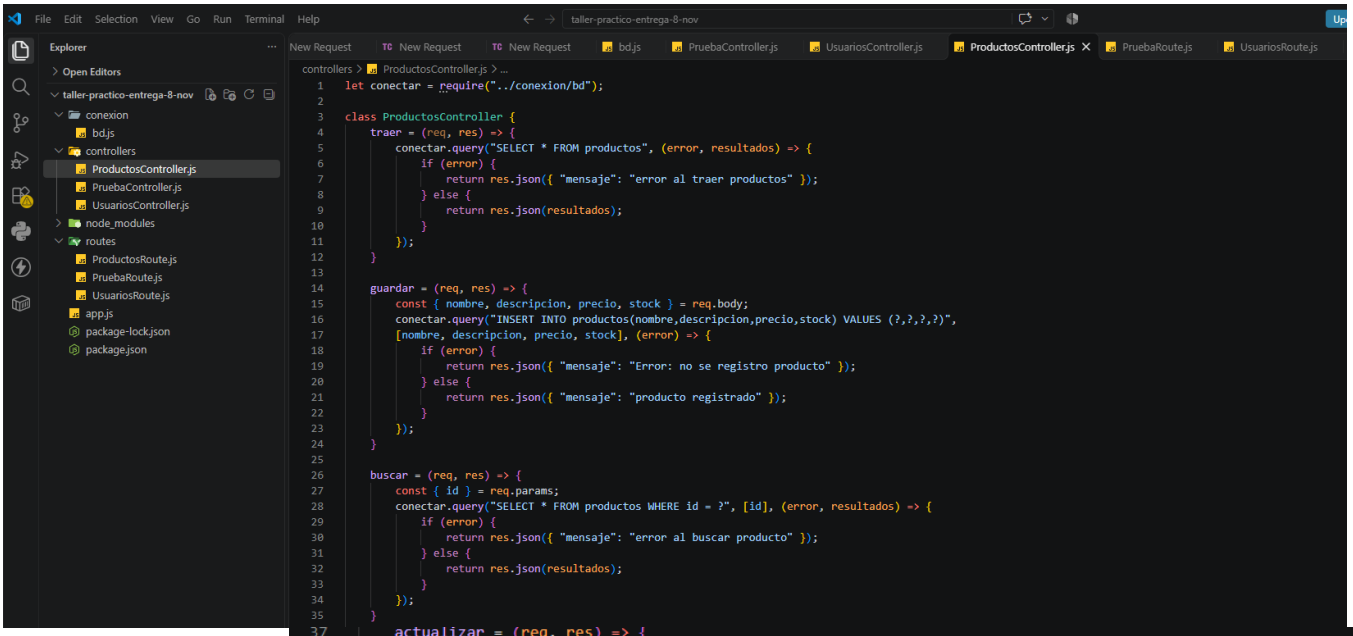

## VISUAL STUDIO CODE

### Conexión
### Bd.js



## Controllers

## ProductosController.js




```
let conectar = require("../conexion/bd");

class ProductosController {
    traer = (req, res) => {
        conectar.query("SELECT * FROM productos", (error, resultados) => {
            if (error) {
                return res.json({ "mensaje": "error al traer productos" });
            } else {
                return res.json(resultados);
            }
        });
    }

    guardar = (req, res) => {
        const { nombre, descripcion, precio, stock } = req.body;
        conectar.query("INSERT INTO productos(nombre,descripcion,precio,stock) VALUES (?,?,?,?)",
        [nombre, descripcion, precio, stock], (error) => {
            if (error) {
                return res.json({ "mensaje": "Error: no se registro producto" });
            } else {
                return res.json({ "mensaje": "producto registrado" });
            }
        });
    }

    buscar = (req, res) => {
        const { id } = req.params;
        conectar.query("SELECT * FROM productos WHERE id = ?", [id], (error, resultados) => {
            if (error) {
                return res.json({ "mensaje": "error al buscar producto" });
            } else {
                return res.json(resultados);
            }
        });
    }
    actualizar = (req, res) => {
        const { id } = req.params;
        const { nombre, descripcion, precio, stock } = req.body;
        conectar.query("UPDATE productos SET nombre = ?, descripcion = ?, precio = ?, stock = ? WHERE id = ?",
        [nombre, descripcion, precio, stock, id], (error) => {
            if (error) {
                return res.json({ "mensaje": "error al actualizar producto" });
            } else {
                return res.json({ "mensaje": "producto actualizado" });
            }
        });
    }

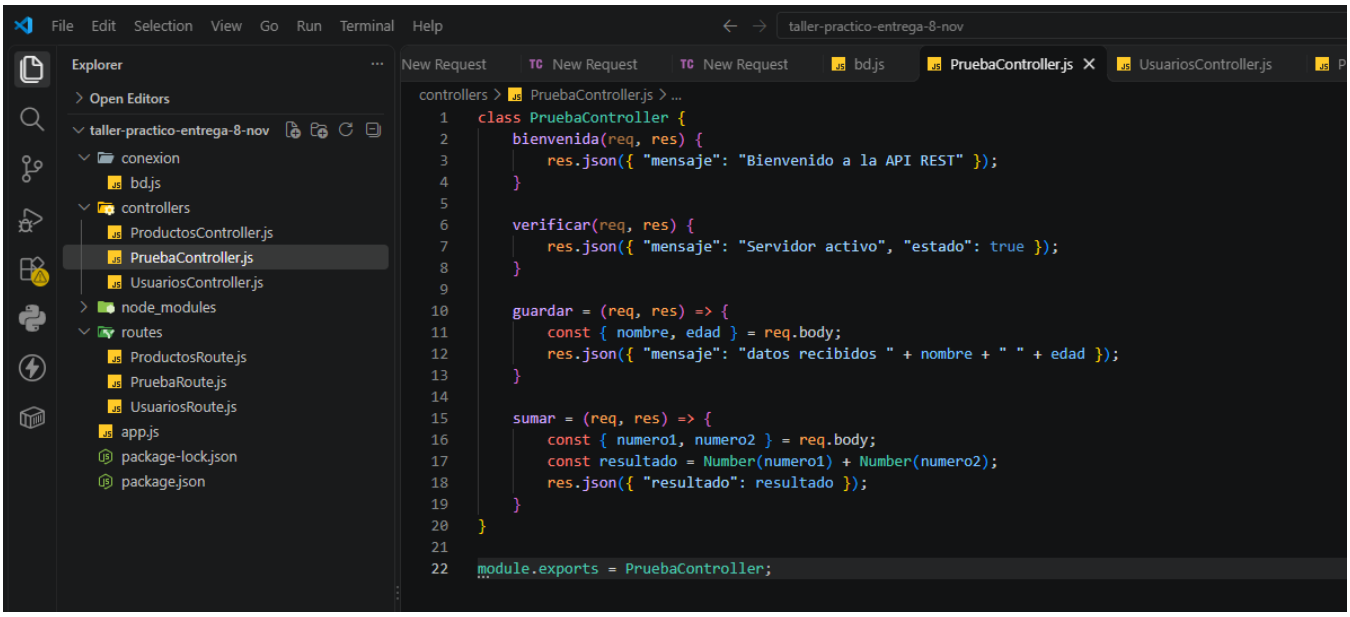
    eliminar = (req, res) => {
        const { id } = req.params;
        conectar.query("DELETE FROM productos WHERE id = ?", [id], (error) => {
            if (error) {
                return res.json({ "mensaje": "error al eliminar producto" });
            } else {
                return res.json({ "mensaje": "producto eliminado" });
            }
        });
    }

    agotados = (req, res) => {
        conectar.query("SELECT * FROM productos WHERE stock = 0", (error, resultados) => {
            if (error) {
                return res.json({ "mensaje": "error al consultar agotados" });
            } else {
                return res.json(resultados);
            }
        });
    }

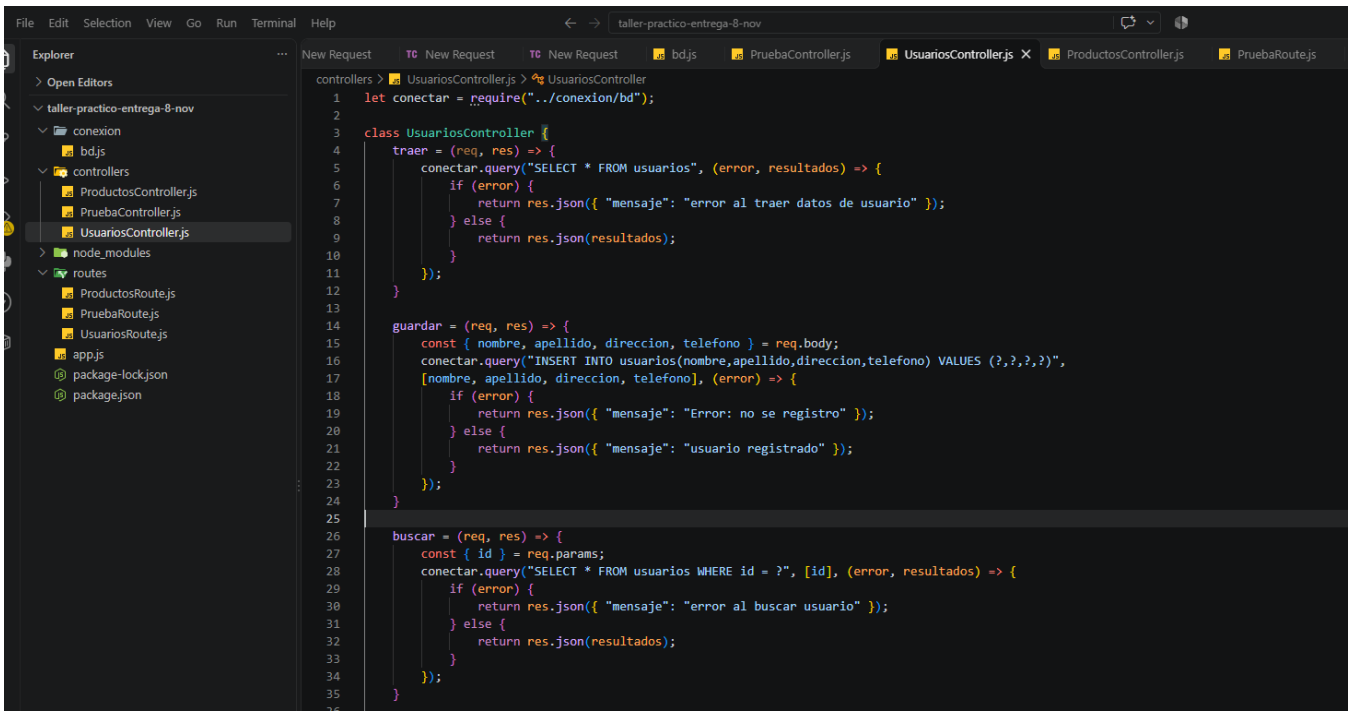
    bajoStock = (req, res) => {
        conectar.query("SELECT * FROM productos WHERE stock < 5", (error, resultados) => {
            if (error) {
                return res.json({ "mensaje": "error al consultar bajo stock" });
            } else {
                return res.json(resultados);
            }
        });
    }
}

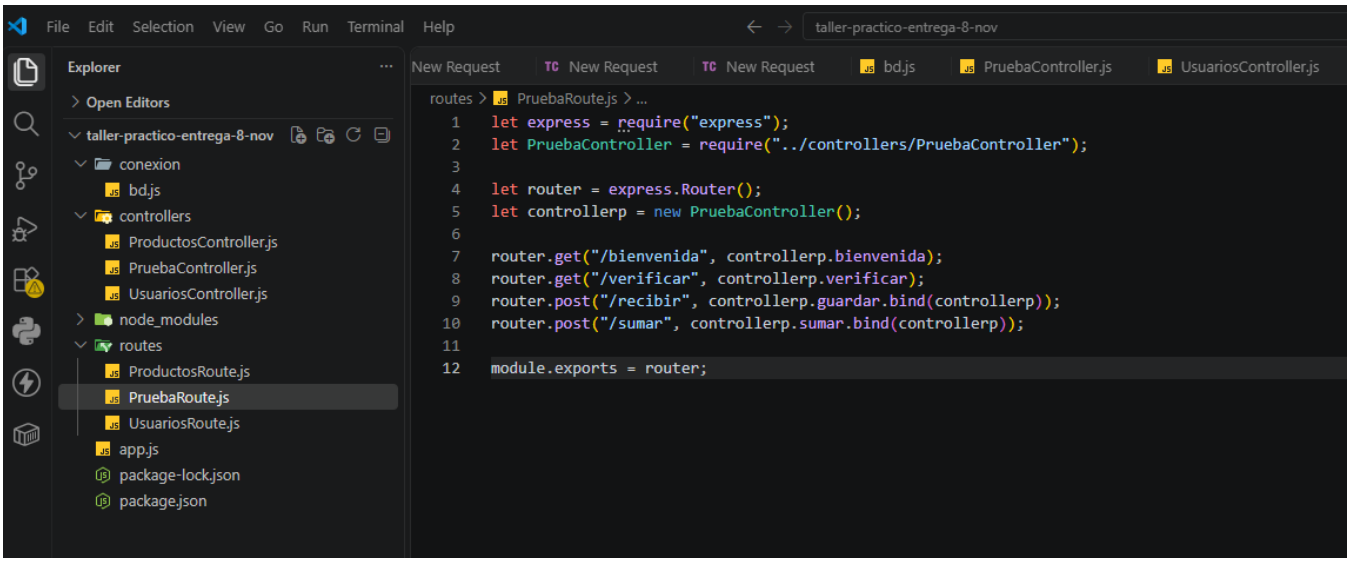
module.exports = ProductosController;
```

## PruebaController.js




## UsuariosController.js



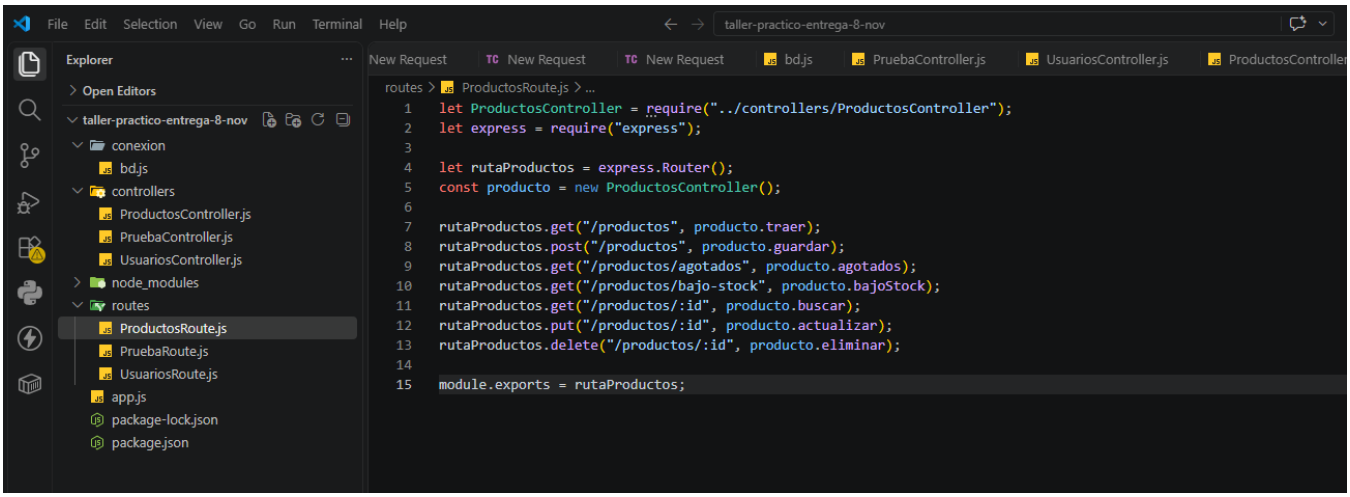



## Routes

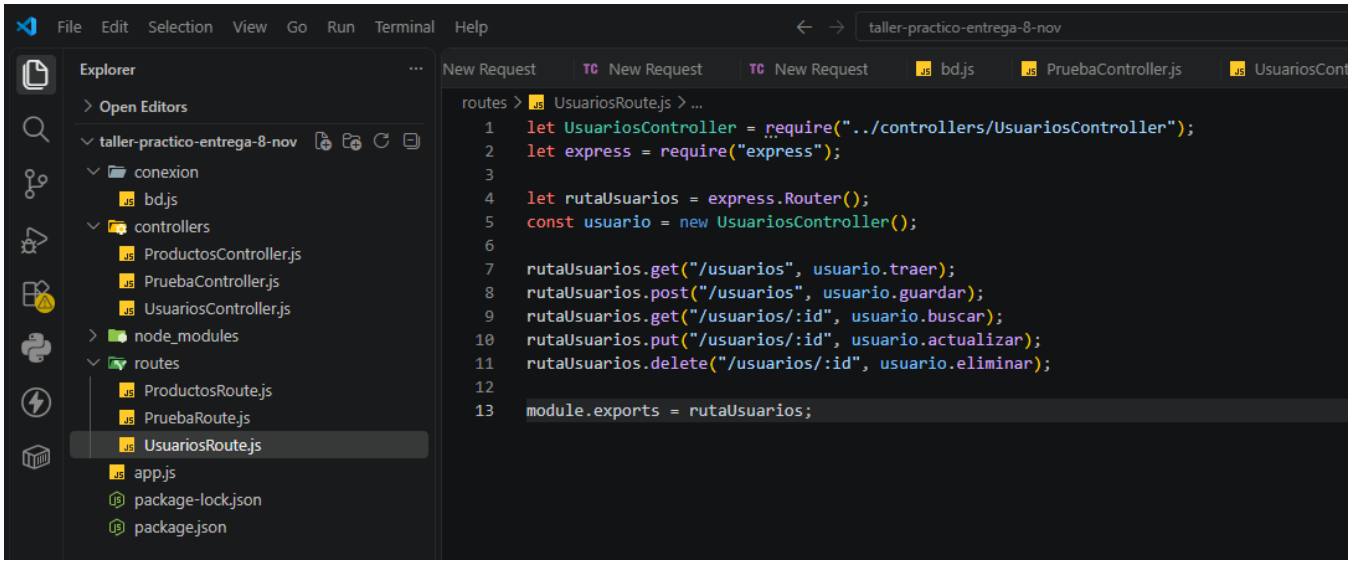
## ProductosRoute.js

```
36
37    actualizar = (req, res) => {
38        const { id } = req.params;
39        const { nombre, apellido, direccion, telefono } = req.body;
40        conectar.query("UPDATE usuarios SET nombre = ?, apellido = ?, direccion = ?, telefono = ? WHERE id = ?",
41        [nombre, apellido, direccion, telefono, id], (error) => {
42            if (error) {
43                return res.json({ "mensaje": "error al actualizar usuario" });
44            } else {
45                return res.json({ "mensaje": "usuario actualizado" });
46            }
47        });
48    }
49
50    eliminar = (req, res) => {
51        const { id } = req.params;
52        conectar.query("DELETE FROM usuarios WHERE id = ?", [id], (error) => {
53            if (error) {
54                return res.json({ "mensaje": "error al eliminar usuario" });
55            } else {
56                return res.json({ "mensaje": "usuario eliminado" });
57            }
58        });
59    }
60 }
61
62 module.exports = UsuariosController;
```

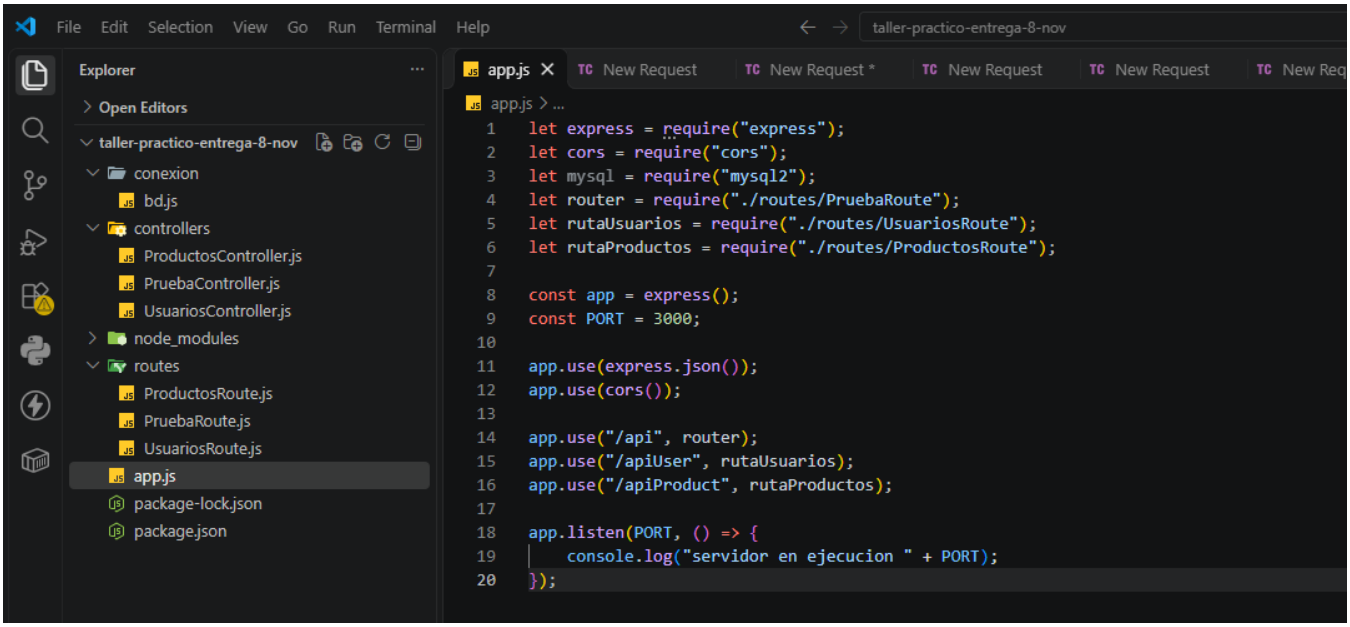
## PruebaRoute.js



## UsuariosRoue.js




## App.js



**MYSQL**

taller_api

Limit to 2000 rows

```sql
CREATE DATABASE IF NOT EXISTS taller_api;
USE taller_api;

CREATE TABLE IF NOT EXISTS usuarios (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    direccion VARCHAR(200) NULL,
    telefono VARCHAR(20) NULL
);
SELECT * FROM usuarios;

CREATE TABLE IF NOT EXISTS productos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);

SELECT * FROM productos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| id | nombre | descripcion | precio | stock |
|---|---|---|---|---|
| 1 | Portátil Lenovo | Core i7, 16 GB RAM | 3500000.00 | 10 |
| NULL | NULL | NULL | NULL | NULL |

## 2. Preguntas de Análisis y Sustentación

### 1. ¿Qué es Node.js y cuál es su función en el proyecto?

Es un entorno de ejecución de JavaScript fuera del navegador que permite ejecutar código del lado del servidor para administrar peticiones HTTP y la conexión a la base de datos.

### 2. ¿Qué es Express y qué ventajas ofrece?

Es un framework para Node.js que simplifica la creación de servidores web, el manejo de rutas y la gestión de middlewares.

### 3. ¿Qué es una API REST?

Es una interfaz que utiliza el protocolo HTTP y verbos estándar (GET, POST, PUT, DELETE) para permitir la comunicación e intercambio de datos en formato JSON entre cliente y servidor.

4. **¿Qué diferencia existe entre GET, POST, PUT y DELETE?**

   GET se usa para consultar recursos, POST para registrar o crear nuevos datos, PUT para actualizar registros existentes y DELETE para borrarlos.

5. **¿Qué función cumple express.json()?**

   Es un middleware encargado de analizar y procesar las solicitudes entrantes que tienen formato JSON, convirtiéndolas en un objeto JavaScript accesible mediante req.body.

6. **¿Qué es una clase en JavaScript?**

   Es una plantilla o molde para la programación orientada a objetos que agrupa propiedades y métodos.

7. **¿Qué es un objeto y cómo se instancia?**

   Es una instancia o representación concreta de una clase que contiene sus comportamientos; se crea utilizando la palabra reservada new (por ejemplo: const usuario = new UsuariosController()).

8. **¿Qué función cumple el constructor?**

   Es un método especial que se ejecuta automáticamente al instanciar una clase para inicializar sus propiedades.

9. **¿Qué diferencia existe entre un método tradicional y una propiedad con función flecha?**

   Las funciones flecha conservan y vinculan automáticamente el contexto de this a la instancia de la clase, evitando que se pierda el contexto al usarlas como callbacks en Express.

10. **¿Para qué se utiliza module.exports y require()?**

    module.exports se utiliza para exportar módulos, clases o funciones desde un archivo, y require() permite importarlos y utilizarlos en otros archivos del proyecto.

11. **¿Qué responsabilidad tiene un Controller y un Routes?**

    Routes define las rutas y los métodos HTTP que direccionan las peticiones, mientras que Controller contiene la lógica de negocio y las consultas hacia la base de datos.

12. **¿Por qué la conexión MySQL se comparte desde app.js?**

Permite centralizar un único punto de configuración y conexión global que puede ser importado y reutilizado por los controladores.

**13. ¿Qué función cumplen los signos ? en las consultas SQL parametrizadas?**

Funcionan como marcadores de posición que ayudan a prevenir ataques de inyección SQL y sanitizan los datos de entrada.

**14. ¿Por qué es importante controlar los errores en una API?**

Para evitar que el servidor se caiga inesperadamente y para devolver respuestas claras y amigables al cliente cuando algo falla.